Module 8: LangChain for LLM Application Development

Course Overview

LangChain has become the de facto standard framework for building production-grade LLM applications. This module provides comprehensive training on leveraging LangChain to rapidly develop, deploy, and scale intelligent applications that combine language models with external tools, knowledge bases, and complex workflows. Students will master the complete LangChain ecosystem—from foundational abstractions through advanced agent patterns—enabling them to build real-world applications efficiently[1] [2]. The module emphasizes practical implementation, production considerations, and best practices from industry leaders.

Table of Contents

1. Introduction to LangChain
2. LangChain Foundations and Core Concepts
3. Benefits and Value Propositions of LangChain
4. Core Components of LangChain
5. Building Blocks: Chains and Prompts
6. Memory Systems and Conversation Management
7. Retrieval-Augmented Generation (RAG) with LangChain
8. LangChain Agents and Tool Integration
9. Off-the-Shelf Chains and Advanced Patterns
10. Hands-on Implementation: Build and Deploy LLM Applications

1. Introduction to LangChain

1.1 What is LangChain?

LangChain is an open-source framework for building LLM-powered applications with composable, modular components[3]. Rather than building monolithic applications, LangChain enables developers to combine smaller, reusable components—language models, prompts, retrievers, memory systems, and tools—into sophisticated workflows.

Core Mission:

Make it easy for developers to build applications powered by language models without getting bogged down by infrastructure details or vendor lock-in.

1.2 The LangChain Ecosystem

LangChain consists of multiple integrated products:

Product	amp; Purpose	amp; Use Case
LangChain Core	amp; Framework for building chains, agents, RAG systems	amp; Development and prototyping
LangSmith	amp; Debugging, evaluation, observability	amp; Production monitoring and improvement
LangGraph	amp; State machine for complex agent workflows	amp; Multi-step autonomous systems
LangServe	amp; REST API deployment	amp; Production deployment

Table 1: LangChain Ecosystem Products

1.3 Market Adoption and Industry Statistics

2024-2025 Adoption:

- **51% of companies** using AI agents in production (LangChain State of AI Agents Report)
- **78% of companies** have active plans to deploy agents soon
- **Mid-sized companies** most aggressive (63% in production)
- Primary frameworks: LangChain, LlamaIndex, AutoGen

Enterprise Usage:

Major companies including Stripe, Notion, DuckDuckGo, and thousands of startups rely on LangChain for production AI applications.

1.4 Why LangChain Matters for Developers

Traditional Approach:

Building LLM applications requires:

- Integrating multiple APIs (OpenAI, Anthropic, Azure, local models)
- Managing prompts and templates
- Building memory systems
- Creating retrieval systems
- Handling errors and retries
- Testing and debugging
- Deploying at scale

Each task requires custom code, increasing complexity and time-to-market.

LangChain Approach:

Pre-built, composable components handle these concerns, allowing developers to focus on application logic.

2. LangChain Foundations and Core Concepts

2.1 Five Core Abstractions

LangChain organizes development around five fundamental abstractions [4]:

1. Models

Language models and other model types (embeddings, classifiers).

- LLMs (Large Language Models): GPT-4, Claude, Gemini, Llama
- Chat Models: Specialized for conversational interfaces
- Embeddings: Convert text to vectors for semantic search

2. Prompts

Instructions to language models, including templates and examples.

- Prompt Templates: Reusable instruction patterns with placeholders
- Few-shot Examples: Examples demonstrating desired behavior
- Chains: Composed prompts for multi-step tasks

3. Output Parsers

Convert unstructured LLM outputs to structured formats (JSON, lists, custom objects).

- JSON Parser: Extract JSON from LLM output
- List Parser: Extract lists from text
- Custom Parsers: Domain-specific structured extraction

4. Retrievers

Fetch relevant information from external knowledge sources.

- Vector Store Retrievers: Semantic search over embeddings
- Web Retrievers: Search the internet
- SQL Retrievers: Query databases
- Hybrid Retrievers: Combine multiple retrieval methods

5. Agents

Autonomous systems that use tools to accomplish tasks.

- Reasoning Loop: Think → Act → Observe → Repeat
- Tool Integration: Connect LLMs to APIs and functions
- State Management: Track agent decisions and actions
- Error Handling: Recover from failures gracefully

2.2 The LangChain Expression Language (LCEL)

LCEL is a declarative approach to composing LangChain components introduced in Q3 2023 [5].

Philosophy:

"Describe what should happen, not how it should happen."

LCEL allows LangChain to optimize execution—streaming, parallelization, error handling—automatically.

Example: Simple Chain

```
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

# Define components
prompt = ChatPromptTemplate.from_template("Explain {concept}")
llm = ChatOpenAI(model="gpt-4")
parser = StrOutputParser()

# Compose with pipe operator
chain = prompt | llm | parser

# Run
result = chain.invoke({"concept": "quantum computing"})
```

Benefits:

- **Readable:** Pipe operator clearly shows data flow
- **Optimizable:** LangChain optimizes execution automatically
- **Streamable:** Results stream as they're generated
- **Observable:** Integrates with LangSmith for debugging

2.3 Design Principles

Modularity:

Every component works independently or combined with others. Use just embeddings without chains, or chains without agents.

Composability:

Components designed to work seamlessly together. Swap vector stores without changing retrieval code.

Standardization:

Consistent interfaces across providers. Switch from OpenAI to Anthropic with minimal code changes.

Provider Agnosticism:

No vendor lock-in. LangChain abstracts over 100+ integrations (LLMs, vector stores, tools, memory).

Example: Model Switching

```
# Start with OpenAI
from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-4")
```

Switch to Anthropic (one line!)

```
from langchain_anthropic import ChatAnthropic
model = ChatAnthropic(model="claude-opus")
```

Same chain still works

```
result = chain.invoke({"concept": "quantum computing"})
```

3. Benefits and Value Propositions of LangChain

3.1 Development Speed and Time-to-Market

Problem: Building LLM apps involves boilerplate (100+ lines per feature).

LangChain Solution: Pre-built components reduce code by 80%+.

Example: RAG System

Traditional approach: 200+ lines (authentication, embeddings, vector store, retrieval, prompting, error handling)

LangChain approach:

```
from langchain.document_loaders import WebBaseLoader
from langchain.embeddings import OpenAIEmbeddings
from langchain.vectorstores import FAISS
from langchain.chains import RetrievalQA
```

Load and index documents

```
loader = WebBaseLoader("https://example.com")
docs = loader.load()
vectorstore = FAISS.from_documents(docs, OpenAIEmbeddings())
```

Create RAG chain

```
qa = RetrievalQA.from_chain_type(
llm=ChatOpenAI(),
chain_type="stuff",
retriever=vectorstore.as_retriever()
)
```

Done! ~30 lines vs 200+

3.2 Flexibility and Extensibility

Built-in Integrations:

LangChain provides integrations for:

- 50+ LLM providers
- 20+ vector databases
- 100+ tools and APIs
- 15+ memory systems
- 30+ document loaders

Custom Components:

Easily extend LangChain with custom:

- Models
- Retrievers
- Tools
- Memory systems
- Output parsers

3.3 Production Readiness

Built-in Production Features:

- **Error Handling:** Automatic retries, timeout management
- **Caching:** Reduce API calls and costs
- **Observability:** LangSmith integration for monitoring
- **Streaming:** Real-time response generation
- **Batch Processing:** Handle multiple requests efficiently
- **Rate Limiting:** Respect API quotas

3.4 Community and Ecosystem

Largest Developer Community:

- 60k+ GitHub stars
- 10k+ contributors
- Thousands of open-source projects built with LangChain
- Extensive documentation and tutorials

Enterprise Adoption:

Major companies choose LangChain, ensuring:

- Long-term support
- Job market opportunities
- Proven patterns and best practices
- Enterprise-grade reliability

3.5 Cost Reduction

Mechanisms:

1. **Reduced Development Time:** Build apps 5-10x faster
2. **Fewer Bugs:** Tested, battle-proven components
3. **Optimized API Calls:** Caching, batching reduce LLM costs 30-50%
4. **Self-Hosted Options:** Avoid vendor lock-in

ROI Example:

Startup building customer support agent:

- Without LangChain: 3 months development, $50k cost

- With LangChain: 2 weeks development, $5k cost
- **Savings: 85% cost, 92% time reduction**

4. Core Components of LangChain

4.1 Language Models

Models are the foundation—all LangChain applications use at least one.

Model Types:

LLMs (Legacy)

Base language models that take text, return text:

```
from langchain_openai import OpenAI

llm = OpenAI(model="gpt-3.5-turbo")
result = llm.invoke("What is AI?")
```

Chat Models (Recommended)

Conversation-optimized, take message lists:

```
from langchain_openai import ChatOpenAI
from langchain_core.messages import HumanMessage, SystemMessage

chat_model = ChatOpenAI(model="gpt-4")
messages = [
SystemMessage(content="You are a helpful assistant."),
HumanMessage(content="What is machine learning?")
]
result = chat_model.invoke(messages)
```

Embeddings

Convert text to dense vectors for semantic search:

```
from langchain_openai import OpenAIEmbeddings

embeddings = OpenAIEmbeddings(model="text-embedding-3-large")
vector = embeddings.embed_query("Hello world")
```

Output: [0.123, -0.456, 0.789, ...]

4.2 Prompt Templates

Structured prompts with placeholders for dynamic content.

Simple Template:

```
from langchain_core.prompts import ChatPromptTemplate

template = ChatPromptTemplate.from_template(
"You are a helpful assistant. "
"Answer this question: {question}"
)
```

```
prompt = template.invoke({"question": "What is Python?"})
```

Few-Shot Prompting:

Include examples demonstrating desired behavior:

```
from langchain_core.prompts.few_shot import FewShotChatMessagePromptTemplate
from langchain_core.prompts import ChatPromptTemplate

examples = [
{"input": "happy", "output": "sad"},
{"input": "tall", "output": "short"},
{"input": "bright", "output": "dark"},
]

example_prompt = ChatPromptTemplate.from_template(
"Input: {input}\nOutput: {output}"
)

few_shot_prompt = FewShotChatMessagePromptTemplate(
examples=examples,
example_prompt=example_prompt,
suffix="Input: {input}\nOutput:",
input_variables=["input"],
)
```

Dynamic Few-Shot Selection:

Select most relevant examples based on query:

```
from langchain_core.prompts import FewShotChatMessagePromptTemplate
from langchain_core.example_selectors import SemanticSimilarityExampleSelector
from langchain_openai import OpenAIEmbeddings
from langchain_community.vectorstores import FAISS
```

Create example selector

```
example_selector = SemanticSimilarityExampleSelector.from_examples(
examples,
OpenAIEmbeddings(),
FAISS,
k=2 # Select top-2 most similar examples
)

few_shot_prompt = FewShotChatMessagePromptTemplate(
example_selector=example_selector,
example_prompt=example_prompt,
suffix="Input: {input}\nOutput:",
input_variables=["input"],
)
```

4.3 Output Parsers

Convert LLM outputs to structured formats.

JSON Parser:

```
from langchain_core.output_parsers import JsonOutputParser
from pydantic import BaseModel, Field

class PersonInfo(BaseModel):
name: str = Field(..., description="Person's name")
age: int = Field(..., description="Person's age")
occupation: str = Field(..., description="Person's job")

parser = JsonOutputParser(pydantic_object=PersonInfo)

prompt = ChatPromptTemplate.from_template(
"Extract person info from: {text}\n{format_instructions}"
).partial(format_instructions=parser.get_format_instructions())

chain = prompt | llm | parser

result = chain.invoke({"text": "John is 30 years old and works as a doctor"})
```

Output: PersonInfo(name='John', age=30, occupation='doctor')

Custom Parser:

```
from langchain_core.output_parsers import BaseOutputParser

class CommaSeparatedListParser(BaseOutputParser):
def parse(self, text: str) -> list:
return text.strip().split(",")

parser = CommaSeparatedListParser()
result = parser.parse("apple, banana, cherry")
```

Output: ['apple', 'banana', 'cherry']

5. Building Blocks: Chains and Prompts

5.1 What Are Chains?

Chains combine multiple components in sequence. The original abstraction in LangChain, now expressed through LCEL.

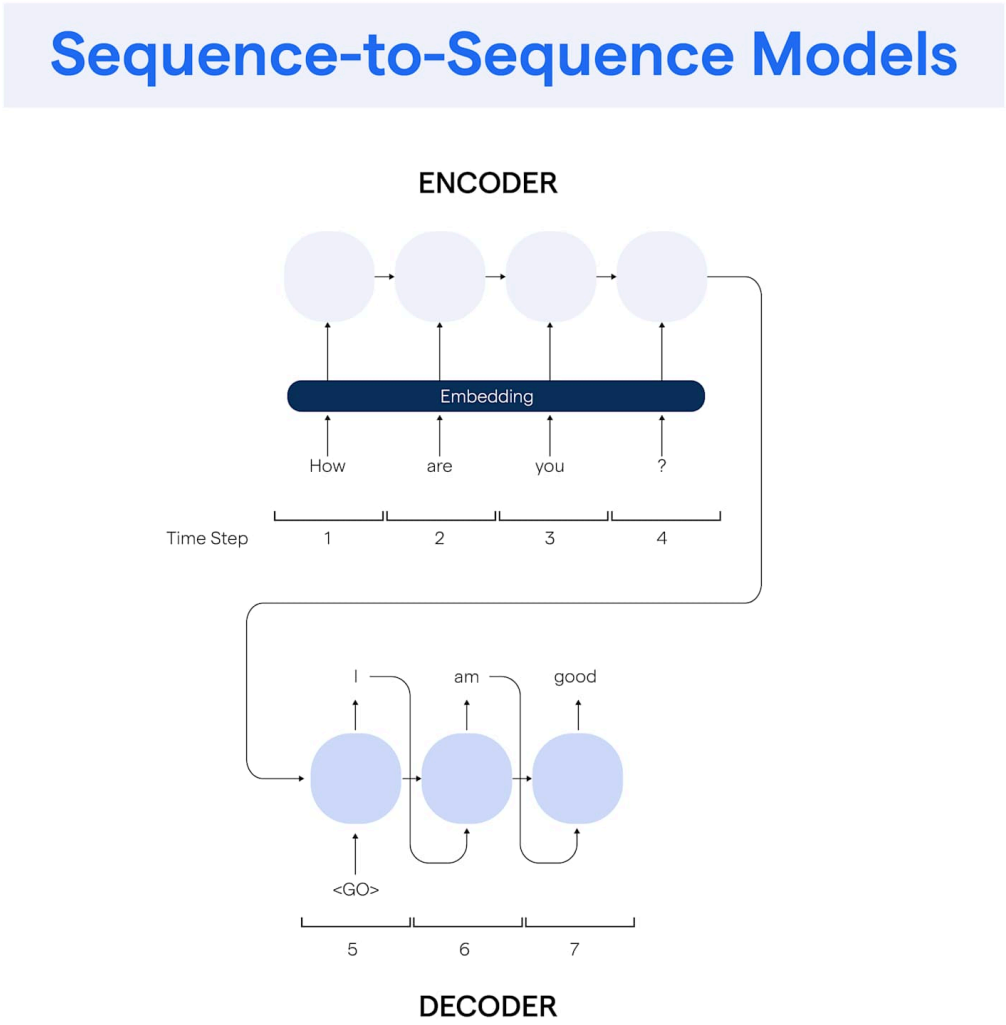

Figure 1: Chain Components: Prompt → Model → Parser forming pipeline

Simple Chain:

1. Take input
2. Pass to prompt template
3. Format with variables
4. Send to LLM
5. Parse output
6. Return result

5.2 Building Chains with LCEL

Example: Multi-Step Chain

```
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
```

Step 1: Generate ideas

```
ideas_prompt = ChatPromptTemplate.from_template(
"Generate 3 creative ideas for: {topic}"
)
```

Step 2: Evaluate ideas

```
eval_prompt = ChatPromptTemplate.from_template(
"Rate these ideas on feasibility (1-10): {ideas}"
)
```

Step 3: Select best

```
select_prompt = ChatPromptTemplate.from_template(
"Pick the most feasible idea and explain why: {ratings}"
)
```

Compose chain

```
llm = ChatOpenAI(model="gpt-4")
parser = StrOutputParser()
```

Chain: Generate → Evaluate → Select

```
idea_chain = ideas_prompt | llm | parser
eval_chain = eval_prompt | llm | parser
select_chain = select_prompt | llm | parser
```

Execute

```
topic = "sustainable energy"
ideas = idea_chain.invoke({"topic": topic})
ratings = eval_chain.invoke({"ideas": ideas})
best_idea = select_chain.invoke({"ratings": ratings})
```

5.3 Off-the-Shelf Chains

LangChain provides pre-built chains for common tasks:

QA Chain:

Answer questions over documents:

```
from langchain.chains import RetrievalQA
from langchain_openai import ChatOpenAI

qa = RetrievalQA.from_chain_type(
llm=ChatOpenAI(),
chain_type="stuff",
retriever=vectorstore.as_retriever()
)

answer = qa.run("What is quantum computing?")
```

Summarization Chain:

```
from langchain.chains import load_summarize_chain

chain = load_summarize_chain(
llm=ChatOpenAI(),
chain_type="map_reduce" # For long documents
)

summary = chain.run(documents)
```

Retrieval-Augmented Generation (RAG):

```
from langchain.chains import create_retrieval_chain
from langchain.chains.combine_documents import create_stuff_documents_chain

qa_prompt = ChatPromptTemplate.from_template(
"Answer based on context: {context}\n\nQuestion: {input}"
)

doc_chain = create_stuff_documents_chain(llm, qa_prompt)
retrieval_chain = create_retrieval_chain(retriever, doc_chain)

result = retrieval_chain.invoke({"input": "What is X?"})
```

5.4 Chain Types and Strategies

Stuff:

Pass all documents directly to LLM.

- Pros: Simple, accurate (full context)
- Cons: Limited by context window
- Use: Small document sets (<20k tokens)

Map-Reduce:

Process documents in parallel, then combine.

- Pros: Handles large document sets
- Cons: More expensive, potential information loss
- Use: Large document collections

Refine:

Iteratively process documents, refining output.

- Pros: Good for long documents
- Cons: Slower (sequential processing)
- Use: High-quality summaries

6. Memory Systems and Conversation Management

6.1 Memory Fundamentals

Memory enables LLMs to maintain context across turns in conversations [6].

Challenge:

LLMs process one request at a time (no inherent memory). Each request resets context unless conversation history is explicitly provided.

LangChain Solution:

Automatic memory management systems that:

1. Store conversation history
2. Retrieve relevant past turns
3. Inject into context window
4. Manage token limits

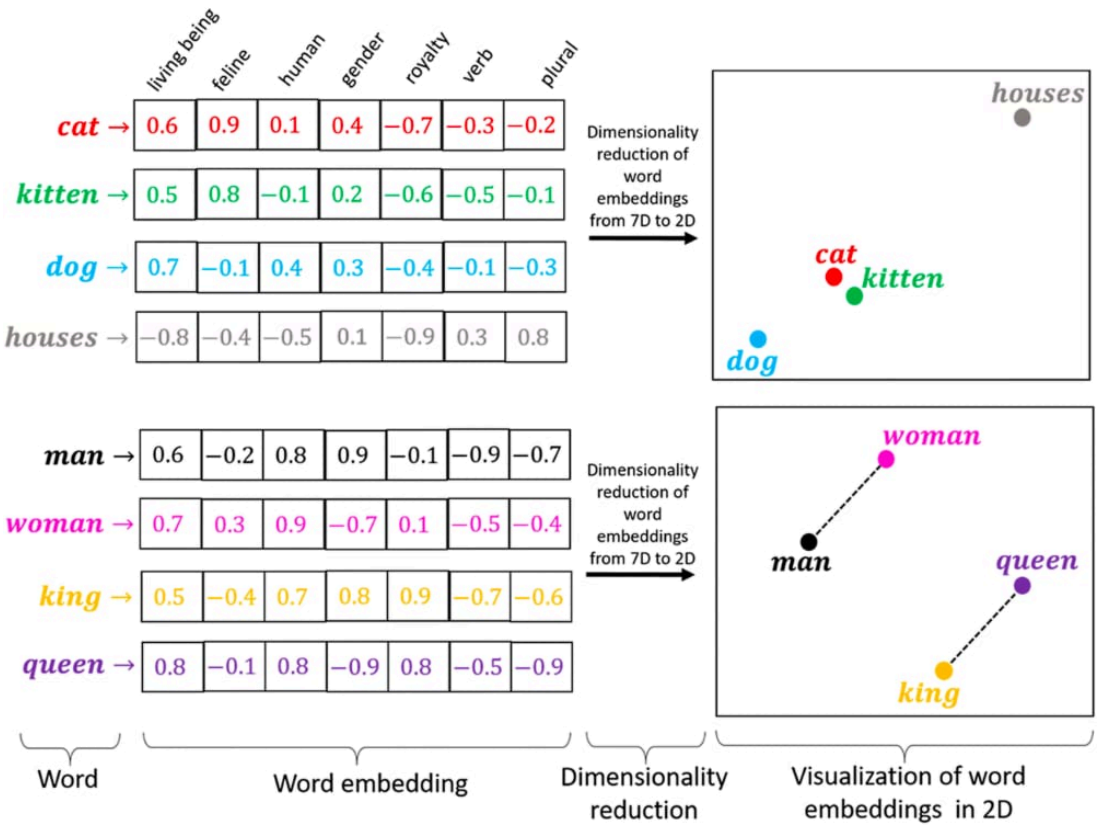

Figure 2: Memory Context: Storing and retrieving past interactions for continuous context

6.2 Memory Types in LangChain

Buffer Memory

Keep all conversation history in memory.

```
from langchain.memory import ConversationBufferMemory

memory = ConversationBufferMemory()
```

Add messages manually

```
memory.save_context(
{"input": "Hi, I'm Alice"},
{"output": "Nice to meet you, Alice!"}
)
```

Or automatically in chain

```
from langchain.chains import ConversationChain

conversation = ConversationChain(
llm=ChatOpenAI(),
memory=ConversationBufferMemory()
)

response = conversation.run("Tell me a joke")
```

Problems:

- Memory grows unbounded
- Eventually exceeds context window
- More expensive (more tokens to process)

Summary Memory

Summarize conversation to reduce token usage.

```
from langchain.memory import ConversationSummaryMemory

memory = ConversationSummaryMemory(
llm=ChatOpenAI(),
buffer="Conversation summary would go here"
)
```

- Pros: Bounded memory, keeps high-level context
- Cons: Details lost, summarization cost
- Use: Long conversations requiring efficiency

Token Buffer Memory

Keep most recent messages within token limit.

```
from langchain.memory import ConversationTokenBufferMemory

memory = ConversationTokenBufferMemory(
llm=ChatOpenAI(),
max_token_limit=2000
)
```

- Pros: Reasonable size, recent context preserved
- Cons: Older context discarded
- Use: Most production conversations

Entity Memory

Track entities (people, places, concepts) across conversation.

from langchain.memory import ConversationEntityMemory

memory = ConversationEntityMemory(llm=ChatOpenAI())

- Pros: Extracts structured information
- Cons: Extraction overhead
- Use: Knowledge-rich conversations

6.3 Custom Memory Systems

Build memory for specific needs:

from langchain_core.memory import BaseMemory
from typing import Any, Dict, List

class CustomMemory(BaseMemory):
messages: List[Dict] = []

```
@property
def memory_variables(self) -&gt; List[str]:
    return ["history"]

def load_memory_variables(self, inputs: Dict[str, Any]) -&gt; Dict[str, str]:
    # Retrieve relevant memories based on input
    relevant = [m for m in self.messages if relevant_check(m)]
    return {"history": "\n".join([m["text"] for m in relevant])}

def save_context(self, inputs: Dict[str, Any], outputs: Dict[str, str]) -&gt; None:
    # Store interaction
    self.messages.append({
        "input": inputs["input"],
        "output": outputs["output"],
        "timestamp": datetime.now()
    })
```

7. Retrieval-Augmented Generation (RAG) with LangChain

7.1 RAG Architecture and Benefits

Retrieval-Augmented Generation (RAG) addresses a key LLM limitation: lack of current knowledge [7].

Problem:

LLMs trained on fixed data (knowledge cutoff). Can't answer questions about:

- Recent events
- Private/proprietary information
- Real-time data (stock prices, weather)
- Specific documents

Solution: RAG

Retrieve relevant information before generating response.

Architecture:

1. **Indexing:** Embed and store documents

2. **Retrieval:** Find relevant documents for query
3. **Augmentation:** Add retrieved docs to prompt
4. **Generation:** LLM generates response grounded in context

7.2 Building RAG Systems with LangChain

Step 1: Load Documents

```
from langchain.document_loaders import PyPDFLoader, WebBaseLoader

# Load from PDF
loader = PyPDFLoader("document.pdf")
docs = loader.load()

# Load from web
web_loader = WebBaseLoader("https://example.com")
web_docs = web_loader.load()
```

Supported Loaders:

- PDFs, Word documents, Markdown
- Web pages, YouTube transcripts
- Notion, Google Drive
- Databases
- 100+ more via integrations

Step 2: Split Documents

```
from langchain.text_splitter import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
chunk_size=1000, # Characters per chunk
chunk_overlap=200, # Overlap to preserve context
separators=["\n\n", "\n", " ", ""] # Split on decreasing granularity
)

chunks = splitter.split_documents(docs)
```

Chunking Strategies:

- **Fixed size:** Simple, but may break mid-sentence
- **Recursive:** Splits hierarchically, preserving structure
- **Semantic:** Groups by meaning (more expensive)

Step 3: Create Embeddings

```
from langchain_openai import OpenAIEmbeddings

embeddings = OpenAIEmbeddings(model="text-embedding-3-large")
```

Embed document chunks

embedded_chunks = embeddings.embed_documents([c.page_content for c in chunks])

Step 4: Store in Vector Database

from langchain_community.vectorstores import FAISS

vectorstore = FAISS.from_documents(chunks, embeddings)

Save for reuse

vectorstore.save_local("vectorstore")

Load

vectorstore = FAISS.load_local("vectorstore", embeddings)

Popular Vector Stores:

Store	Best For	Scale
FAISS	Development, local	Millions
Pinecone	Production, cloud	Billions
Weaviate	Semantic search, hybrid	Billions
Chroma	Simple local search	Millions
Milvus	Enterprise, self-hosted	Billions

Step 5: Create Retriever

retriever = vectorstore.as_retriever(
search_type="similarity", # or "mmr", "similarity_score_threshold"
search_kwargs={"k": 3} # Return top-3 chunks
)

Retrieval Methods:

- **Similarity:** Return most similar chunks
- **MMR (Maximal Marginal Relevance):** Similar + diverse
- **Threshold:** Only return if similarity > threshold

Step 6: Build RAG Chain

from langchain.chains import create_retrieval_chain
from langchain.chains.combine_documents import create_stuff_documents_chain
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate

Define prompt

qa_prompt = ChatPromptTemplate.from_template("""
Answer the question based only on the provided context.
If answer not in context, say "I don't know."

Context: {context}

```
Question: {input}
Answer:
""")
```

Create chain

```
llm = ChatOpenAI(model="gpt-4")
doc_chain = create_stuff_documents_chain(llm, qa_prompt)
rag_chain = create_retrieval_chain(retriever, doc_chain)
```

Use

```
result = rag_chain.invoke({"input": "What is X?"})
print(result["answer"])
```

7.3 Advanced RAG Techniques

Parent-Child Retrieval

Index small chunks for search, but return larger parent documents[8]:

```
from langchain.retrievers import ParentDocumentRetriever
from langchain.storage import InMemoryStore
```

Create child chunks (small, for search)

```
child_splitter = RecursiveCharacterTextSplitter(chunk_size=400)
```

Create parent chunks (large, for context)

```
parent_splitter = RecursiveCharacterTextSplitter(chunk_size=2000)
```

Use retriever

```
parent_doc_retriever = ParentDocumentRetriever(
child_splitter=child_splitter,
parent_splitter=parent_splitter,
vectorstore=vectorstore,
storage=InMemoryStore()
)
```

- **Benefit:** Accurate retrieval (small chunks) + rich context (large parent)
- **Use:** Long documents where exact granularity matters

Ensemble Retrieval

Combine multiple retrieval methods:

```
from langchain.retrievers import EnsembleRetriever

bm25_retriever = BM25Retriever.from_documents(chunks)
vector_retriever = vectorstore.as_retriever()

ensemble = EnsembleRetriever(
retrievers=[bm25_retriever, vector_retriever],
weights=[0.3, 0.7] # BM25: 30%, Vector: 70%
)
```

- **Benefit:** Hybrid search combining keyword + semantic
- **Use:** Production systems requiring robustness

Hypothetical Document Embeddings (HyDE)

Generate hypothetical document and search for similar:

```
from langchain.chains import LLMChain
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate

# Generate hypothetical doc
hyde_prompt = ChatPromptTemplate.from_template(
"Given question: {question}\n"
"Write a hypothetical document that would answer it:"
)

llm = ChatOpenAI()
hyde_chain = LLMChain(llm=llm, prompt=hyde_prompt)

# Generate and retrieve
hypothetical = hyde_chain.run(question=user_question)
results = vectorstore.similarity_search(hypothetical)
```

- **Benefit:** Better retrieval for rare queries
- **Limitation:** Extra LLM call increases latency/cost

8. LangChain Agents and Tool Integration

8.1 Agent Architecture

Agents automate complex tasks by reasoning through steps, using tools as needed [9].

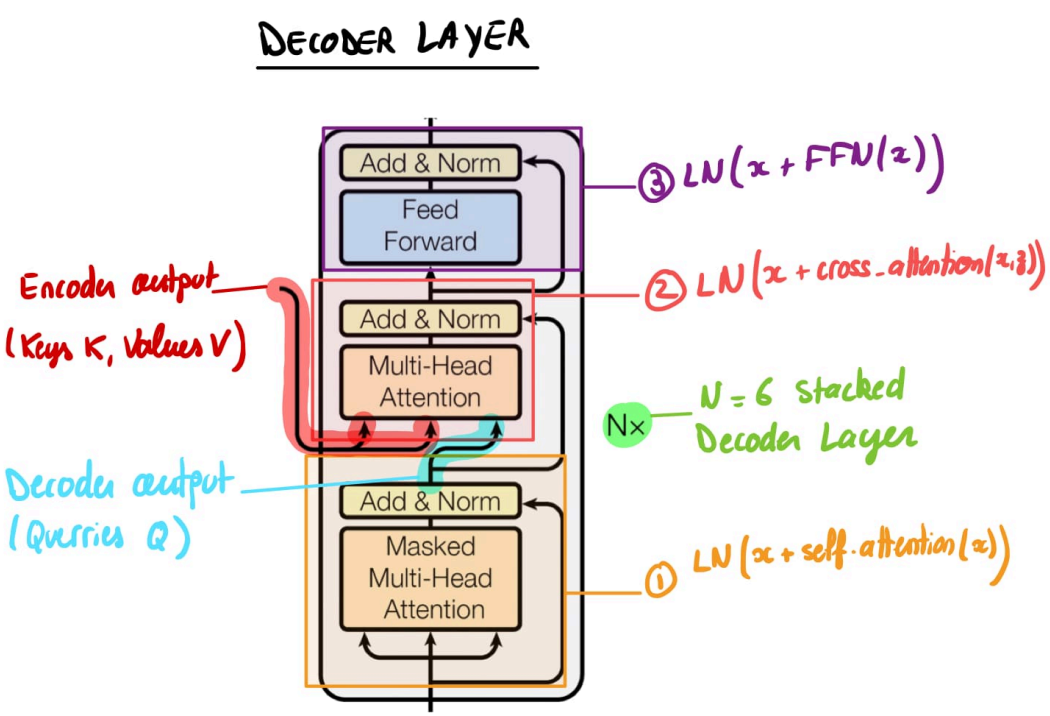

Figure 3: Agent Loop: Reason → Select Tool → Execute → Observe → Repeat

Core Loop:

1. **Observation:** Receive input and context
2. **Thought:** LLM thinks about next step
3. **Action:** Choose tool and arguments
4. **Observation:** Execute tool, get result
5. **Repeat:** Loop until done

8.2 Building Agents

Step 1: Define Tools

```
from langchain.tools import tool
import math

@tool
def calculate_area(radius: float) -> float:
"""Calculate circle area from radius."""
return math.pi * radius ** 2

@tool
def search_web(query: str) -> str:
"""Search the web for information."""
# Implementation...
return results

tools = [calculate_area, search_web]
```

Structured Tool Definition:

```
from langchain.tools import Tool

def get_weather(location: str) -> str:
# Implementation
return "75°F, Sunny"

weather_tool = Tool(
name="get_weather",
func=get_weather,
description="Get current weather for a location"
)
```

Step 2: Create Agent

```
from langchain.agents import create_react_agent
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate

llm = ChatOpenAI(model="gpt-4", temperature=0)
```

Create agent

```
agent = create_react_agent(
llm=llm,
tools=tools,
prompt=prompt # Custom or use default
)
```

Step 3: Execute Agent

```
from langchain.agents import AgentExecutor

executor = AgentExecutor(
agent=agent,
tools=tools,
verbose=True, # Print reasoning
max_iterations=10, # Prevent infinite loops
handle_parsing_errors=True
)
```

Run

```
result = executor.invoke({
"input": "What is the area of a circle with radius 5?"
})

print(result["output"])
```

8.3 Agent Types

ReAct Agent

Reasoning + Acting, alternating:

1. Think about approach
2. Act (use tool)
3. Observe result
4. Repeat

Most common, good balance of reasoning and reliability.

Plan-and-Execute Agent

Plan first, then execute steps:

1. Plan: Break down task into steps
2. Execute: Carry out each step
3. Observe: Verify completion
4. Adapt: Adjust if needed

Better for complex multi-step tasks.

8.4 Tool Integration

Built-in Tools:

LangChain provides integrations for:

- **APIs:** Google Search, Wikipedia, ArXiv, Reddit
- **Web:** WebBrowser tool for reading websites
- **Computation:** Python REPL, Calculator
- **Data:** SQL database queries
- **Documents:** PDF readers, document search

Example: Web Search Agent

```
from langchain.agents import Tool
from langchain.utilities import GoogleSearchAPIWrapper

search = GoogleSearchAPIWrapper()

search_tool = Tool(
name="Google Search",
func=search.run,
description="Search Google for recent information"
)

agent = create_react_agent(
llm=ChatOpenAI(),
tools=[search_tool],
)
# Ask about recent events
result = executor.invoke({
"input": "What major AI models were released in 2024?"
})
```

8.5 Multi-Agent Systems

Coordinate multiple specialized agents:

```
from langchain.agents import AgentExecutor
from langchain.experimental.plan_and_execute import PlanAndExecute
# Define specialized agents
research_agent = create_agent("research_expert", research_tools)
analysis_agent = create_agent("data_analyst", analysis_tools)
writer_agent = create_agent("writer", writing_tools)
# Coordinate
multi_agent = PlanAndExecute(
planner=research_agent,
executor=analysis_agent,
synthesizer=writer_agent
)
```

Handle complex tasks

```
result = multi_agent.run(
"Research latest AI trends and write a comprehensive report"
)
```

9. Off-the-Shelf Chains and Advanced Patterns

9.1 Pre-built Chains

LangChain provides ready-to-use chains for common tasks:

QA Over Documents:

```
from langchain.chains import RetrievalQA

qa = RetrievalQA.from_chain_type(
llm=ChatOpenAI(),
chain_type="stuff",
retriever=vectorstore.as_retriever()
)
```

Summarization:

```
from langchain.chains import load_summarize_chain

chain = load_summarize_chain(
ChatOpenAI(),
chain_type="map_reduce"
)

summary = chain.run(docs)
```

Question Generation:

```
from langchain.chains import QAGenerationChain

qa_gen_chain = QAGenerationChain.from_llm_and_documents(
llm=ChatOpenAI(),
documents=docs
)

qa_pairs = qa_gen_chain.invoke({"docs": docs})
```

9.2 LangGraph for Complex Workflows

LangGraph enables building state machines for sophisticated agent workflows [10].

Example: Multi-Step Customer Support

```
from langgraph.graph import StateGraph, END
from typing import TypedDict

class State(TypedDict):
messages: list
classification: str
ticket_id: str
resolved: bool
```

Define workflow

```
workflow = StateGraph(State)
```

Add nodes

```
workflow.add_node("classify", classify_intent)
workflow.add_node("retrieve", retrieve_kb)
workflow.add_node("generate_response", generate_response)
workflow.add_node("escalate", escalate_to_human)
```

Add edges

```
workflow.add_edge("classify", "retrieve")
workflow.add_conditional_edges(
"retrieve",
lambda s: "generate_response" if s["resolved"] else "escalate",
)
workflow.add_edge("generate_response", END)
workflow.add_edge("escalate", END)

workflow.set_entry_point("classify")
```

Compile and run

```
app = workflow.compile()
result = app.invoke({"messages": [initial_message]})
```

Benefits:

- Visual workflow representation
- State management across steps
- Conditional branching
- Error recovery
- Human-in-the-loop integration

10. Hands-on Implementation: Build and Deploy LLM Applications

10.1 Complete RAG Application

Step 1: Project Setup

```
pip install langchain langchain-openai langchain-community
faiss-cpu python-dotenv
```

Step 2: Build RAG System

```
from langchain.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_openai import OpenAIEmbeddings, ChatOpenAI
from langchain_community.vectorstores import FAISS
from langchain.chains import create_retrieval_chain
from langchain.chains.combine_documents import create_stuff_documents_chain
```

```
from langchain_core.prompts import ChatPromptTemplate
from dotenv import load_dotenv

load_dotenv()
```

Load and process documents

```
loader = PyPDFLoader("research_paper.pdf")
documents = loader.load()

splitter = RecursiveCharacterTextSplitter(
chunk_size=1000,
chunk_overlap=200,
)
chunks = splitter.split_documents(documents)
```

Create embeddings and vector store

```
embeddings = OpenAIEmbeddings()
vectorstore = FAISS.from_documents(chunks, embeddings)
```

Create retriever

```
retriever = vectorstore.as_retriever(
search_type="similarity",
search_kwargs={"k": 3}
)
```

Create QA chain

```
qa_prompt = ChatPromptTemplate.from_template("""
Answer the question based on the provided context only.

Context: {context}

Question: {input}

Answer:
""")

llm = ChatOpenAI(model="gpt-4", temperature=0)
doc_chain = create_stuff_documents_chain(llm, qa_prompt)
rag_chain = create_retrieval_chain(retriever, doc_chain)
```

Test

```
response = rag_chain.invoke({"input": "What are the key findings?"})
print(response["answer"])
```

10.2 Customer Support Agent

```
from langchain.agents import create_react_agent, AgentExecutor
from langchain_openai import ChatOpenAI
from langchain.tools import tool
from langchain_core.prompts import ChatPromptTemplate
```

```
@tool
def search_knowledge_base(query: str) -> str:
"""Search knowledge base for relevant articles."""
# Implementation: query vector DB
return relevant_articles

@tool
def create_ticket(issue: str, priority: str) -> str:
"""Create support ticket for escalation."""
# Implementation: ticket system
return f"Ticket #{ticket_id} created"

@tool
def get_faq(topic: str) -> str:
"""Retrieve FAQ for common topics."""
faqs = {
"billing": "Check our billing FAQ...",
"account": "For account issues...",
"technical": "Technical support guide..."
}
return faqs.get(topic, "No FAQ found")
```

Create agent

```
tools = [search_knowledge_base, create_ticket, get_faq]

prompt = ChatPromptTemplate.from_template("""
You are a helpful customer support agent.
Use available tools to help customers.
If you can't resolve, create a ticket.

{chat_history}
User: {input}
""")

llm = ChatOpenAI(model="gpt-4", temperature=0)
agent = create_react_agent(llm, tools, prompt)

executor = AgentExecutor(
agent=agent,
tools=tools,
verbose=True,
max_iterations=5
)
```

Test

```
response = executor.invoke({
"input": "I'm having billing issues",
"chat_history": ""
})
```

10.3 Deploy with LangServe

app.py

```
from fastapi import FastAPI
from langserve import add_routes
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
```

Create chain

```
template = "You are a helpful assistant. {query}"
prompt = ChatPromptTemplate.from_template(template)
llm = ChatOpenAI()
chain = prompt | llm
```

Create FastAPI app

```
app = FastAPI()
```

Add LangChain routes

```
add_routes(
app,
chain,
path="/chain",
)
```

Run: langchain serve

Client usage

```
from langserve import RemoteRunnable

remote_chain = RemoteRunnable("http://localhost:8000/chain/")
response = remote_chain.invoke({"query": "What is AI?"})
```

10.4 Production Best Practices

Error Handling:

```
from langchain.schema import OutputParserException

try:
result = chain.invoke({"input": user_input})
except OutputParserException as e:
logger.error(f"Parse error: {e}")
result = {"error": "Invalid response format"}
except Exception as e:
logger.error(f"Unexpected error: {e}")
result = {"error": "System error"}
```

Monitoring with LangSmith:

```
from langsmith import Client
```

client = Client()

Traces automatically collected

with client.trace("my_chain") as trace:
result = my_chain.invoke({"input": "test"})
trace.log_metrics({"latency": elapsed_time})

Caching:

from langchain.cache import RedisCache
import redis

redis_client = redis.from_url("redis://localhost")
langchain.llm_cache = RedisCache(redis_client)

Repeated queries now cached

Conclusion and Future Directions

LangChain has become the standard framework for LLM application development, enabling rapid, reliable AI system development.

Key Takeaways:

- LangChain provides modular, composable components for LLM applications
- LCEL declarative approach enables readable, optimizable chains
- RAG systems ground LLMs in current knowledge
- Agents automate complex tasks through reasoning and tool use
- Production features (caching, monitoring, error handling) ensure reliability
- Multi-agent systems coordinate specialized agents for complex tasks

Emerging Directions:

- **Agentic Workflows:** More sophisticated multi-agent coordination
- **LangGraph Adoption:** State machines for complex agent logic
- **Observability:** LangSmith for production monitoring and optimization
- **Specialized Models:** Domain-specific agents and tools
- **Hybrid RAG:** Combining multiple retrieval strategies
- **Autonomous Systems:** Agents that improve over time through feedback

LangChain continues rapid evolution—practitioners must stay current with releases and community patterns. The framework's flexibility ensures it can adapt to new AI capabilities and paradigms.

References

[1] Harrison, C. (2024). LangChain: A framework for developing applications powered by language models. *GitHub Repository*, https://github.com/langchain-ai/langchain.

[2] LangChain Team. (2025). State of AI Agents Report. *LangChain Blog*.

[3] LangChain Documentation. (2025). Introduction and concepts. Retrieved from https://python.langchain.com/docs.

[4] DEV Community. (2025). The complete beginner's guide to LangChain: Core abstractions explained. *DEV to Guides*.

[5] LangChain Team. (2023). Introducing LangChain Expression Language (LCEL). *LangChain Blog*, Q3 2023.

[6] Wiegreffe, E., et al. (2024). Memory in large language models: A survey. *arXiv preprint arXiv:2404.05254*.

[7] Lewis, P., et al. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *NeurIPS*.

[8] Wang, L., et al. (2023). Knowledge-augmented language model and its application to unsupervised named-entity recognition. *ACL*.

[9] Yao, S., et al. (2022). ReAct: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*.

[10] LangChain Team. (2024). LangGraph: Agentic workflow framework. *GitHub*, https://github.com/langchain-ai/langgraph.

⁂

1. https://www.langchain.com
2. https://dev.to/fonyuygita/the-complete-beginners-guide-to-langchain-why-every-developer-needs-this-framework-in-2025part-1-2d55
3. https://www.langchain.com/stateofaiagents
4. https://blog.langchain.com/langchain-state-of-ai-2024/
5. https://www.pingcap.com/article/step-by-step-guide-to-using-langchain-for-ai-projects/
6. https://pub.towardsai.net/introduction-to-retrieval-augmented-generation-rag-using-langchain-and-lamaindex-bd0047628e2a
7. https://www.reddit.com/r/LangChain/comments/1gxnse4/how_are_you_deploying_your_agents_in_production/
8. https://github.com/langchain-ai/langchain
9. https://www.leoniemonigatti.com/blog/retrieval-augmented-generation-langchain.html
10. https://www.langchain.com/langgraph